XTART

FORMACIÓN PROFESIONAL EN TECNOLOGÍA

HIDDEN STUDIO

PROYECTO FINAL 2º DAM

SHIHUI CHEN

ÍNDICE

INTRODUCCIÓN 4
IDEA DE NEGOCIO 5
SGE – SISTEMA DE GESTIÓN EMPRESARIAL 6
ARQUITECTURA GENERAL DEL SISTEMA 8
COMPARACIÓN 9
BASE DE DATOS 10
OPERACIONES CRUD Y CONSULTAS 11
Acceso a la base de datos 13
PSP & ACCESO A DATOS (BACKEND) 14
CRUD 24
DESARROLLO DE INTERFACES – ANGULAR (FRONTEND) 28
Guards e interceptors 29
Models 30
Services 30
Dashboard 31
Games 32
Consumo de API externa 34
PMDM – VIDEOJUEGO: STAB-CORE (UNITY) 37
INTEGRACIÓN WEB + UNITY 42
CONCLUSIÓN 47
Readme Backend 49
Readme Frontend 54
Enlace al repositorio 55

INTRODUCCIÓN

En este proyecto se desarrolla Hidden Studio, un estudio independiente de desarrollo de videojuegos acompañado de una plataforma web propia. El objetivo principal es diseñar e implementar una solución completa que permita la gestión de usuarios, la distribución de videojuegos y la base para la creación de una comunidad de jugadores.

Describiremos siguientemente la idea de negocio, el sistema de gestión empresarial, las decisiones técnicas adoptadas y cómo hemos desarrollado el frontend, backend, diseño y videojuego de nuestro proyecto.

IDEA DE NEGOCIO

La idea de negocio de Hidden Studio se basa en la creación de un estudio independiente de desarrollo de videojuegos que, además de producir sus propios títulos, ofrece una plataforma web propia de distribución y comunidad. Esta plataforma permitirá a los usuarios registrarse e iniciar sesión para acceder a los videojuegos desarrollados por el estudio, consultar información personalizada de su perfil (como días registrados, descripciones o actividad) y, en un futuro, acceder a nuevas funcionalidades relacionadas con la comunidad y la experiencia del jugador.

Hidden Studio nace como respuesta al crecimiento del mercado indie y a la dificultad que tienen los pequeños estudios para destacar en plataformas masivas como Steam. A diferencia de estas grandes plataformas, el proyecto propone un entorno más ligero, cercano y especializado, centrado exclusivamente en los juegos del propio estudio, lo que permite un mayor control sobre la distribución, la experiencia del usuario y el análisis de estadísticas de los jugadores.

El modelo de negocio se apoyará en varias fuentes de ingresos: descargas premium, publicidad integrada en la plataforma y futuras suscripciones que ofrecerán ventajas adicionales a los usuarios. La infraestructura técnica estará basada en una arquitectura cliente-servidor, con un frontend desarrollado en Angular, un backend en Spring Boot, una base de datos MongoDB y videojuegos desarrollados en Unity, que constituyen el producto principal del negocio.

La propuesta de valor de Hidden Studio se centra en ofrecer una experiencia personalizada, una integración directa entre la plataforma y los juegos desarrollados en Unity y un espacio donde los jugadores puedan sentirse parte de una comunidad más reducida y especializada. El público objetivo serán principalmente jugadores casuales interesados en videojuegos indie y experiencias diferentes a las ofrecidas por los grandes estudios.

SGE – SISTEMA DE GESTIÓN EMPRESARIAL

Hidden Studio nace ante la necesidad de ofrecer videojuegos independientes (indie) con una identidad propia, apoyados por una plataforma web que centralice la gestión de usuarios, perfiles, progresos y distribución de contenido. Existe una demanda creciente de experiencias de juego originales y de comunidades online donde los jugadores puedan interactuar con los creadores y entre ellos.

Hidden Studio se organizará en diferentes áreas funcionales, permitiendo una gestión eficiente del proyecto y un desarrollo escalable del negocio.

Para el estudio, la infraestructura tecnológica será ideal poder gestionar usuarios y autenticación de forma segura, almacenar progresos, estadísticas y datos asociados a los videojuegos y facilitar la publicación y actualización de contenidos de manera centralizada. Por tanto, la empresa contará con una estructura organizativa dividida en áreas:

- Desarrollo de videojuegos, encargada del diseño, programación y mantenimiento de los juegos creados con Unity.
- Backend, responsable de la lógica del servidor, la seguridad, la gestión de usuarios y la conexión con la base de datos MongoDB.
- Frontend, dedicada al desarrollo de la interfaz web con Angular, asegurando una experiencia de usuario clara, accesible y atractiva.
- Diseño, que abarcará el diseño visual, la identidad gráfica y los elementos artísticos tanto de la plataforma como de los videojuegos.

Los flujos de trabajo ágiles establecido permiten iterar rápidamente en el desarrollo de videojuegos y en la mejora de la plataforma web. Se priorizará la seguridad de los datos de los usuarios, la estabilidad del sistema y la escalabilidad del servicio para adaptarse al crecimiento del número de jugadores.

Desde el punto de vista tecnológico, Hidden Studio utilizará herramientas software como Unity para el desarrollo de videojuegos y frameworks web modernos para la creación de la plataforma.

El backend estará desarrollado con Spring Boot y se ejecutará en entorno local, utilizando una base de datos MongoDB, lo que permite un desarrollo controlado, seguro y adecuado a las necesidades de nuestro proyecto. Esta configuración facilita las pruebas, el mantenimiento y la demostración del funcionamiento completo del sistema sin necesidad de infraestructura externa.

No obstante, el diseño de la arquitectura se ha planteado de forma que, en un futuro, el sistema pueda migrarse fácilmente a un entorno en la nube, permitiendo mayor escalabilidad, disponibilidad y crecimiento del número de usuarios.

Como competencia, Hidden Studio compite indirectamente con:

- Estudios de desarrollo indie que publican sus juegos en plataformas externas.
- Plataformas de distribución digital que ofrecen servicios de comunidad y perfiles de usuario.

Pero nuestra diferenciación principal radica en la creación de un entorno propio donde los videojuegos del estudio se integran directamente con la plataforma web, permitiendo una experiencia más cohesionada y control total sobre los datos, la evolución del sistema y la relación con los usuarios.

ARQUITECTURA GENERAL DEL SISTEMA

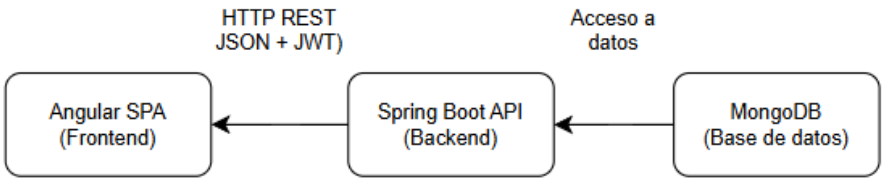
La arquitectura general del sistema se basa en un modelo cliente-servidor, donde el frontend y el backend están claramente separados y se comunican mediante una API REST.

El frontend consiste en una aplicación web de tipo Single Page Application (SPA) desarrollada con Angular. Esta aplicación se encarga de la interacción con el usuario, la gestión de vistas y el consumo de los servicios expuestos por el backend.

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP REST, utilizando JSON como formato de intercambio de datos. Para garantizar la seguridad y el control de acceso, se emplea un sistema de autenticación y autorización basado en JSON Web Tokens (JWT).

El backend está desarrollado con Spring Boot y expone una API REST que centraliza toda la lógica de negocio del sistema. Este backend se encarga de la gestión de usuarios, la seguridad, el acceso a los datos y la validación de las operaciones realizadas desde el cliente. A nivel estructural, sigue una arquitectura monolítica en capas, separando claramente las responsabilidades en capas de controlador, servicio y acceso a datos, lo que facilita el mantenimiento y la comprensión del sistema.

Finalmente, la base de datos MongoDB se utiliza para el almacenamiento persistente de la información, permitiendo una gestión flexible de los datos del sistema.



Este diagrama se representa la arquitectura cliente-servidor del sistema, donde el frontend Angular consume la API REST desarrollada con Spring Boot, que a su vez gestiona el acceso a la base de datos MongoDB.

COMPARACIÓN

Durante el diseño del sistema se han valorado diferentes enfoques arquitectónicos, entre ellos la arquitectura de microservicios y la arquitectura monolítica en capas.

La arquitectura de microservicios ofrece ventajas como la escalabilidad independiente y el despliegue distribuido. Sin embargo, este enfoque resulta excesivamente complejo para nuestro proyecto, ya que requiere una infraestructura avanzada, mayor gestión de comunicaciones entre servicios y un esfuerzo de desarrollo y mantenimiento demasiado alto.

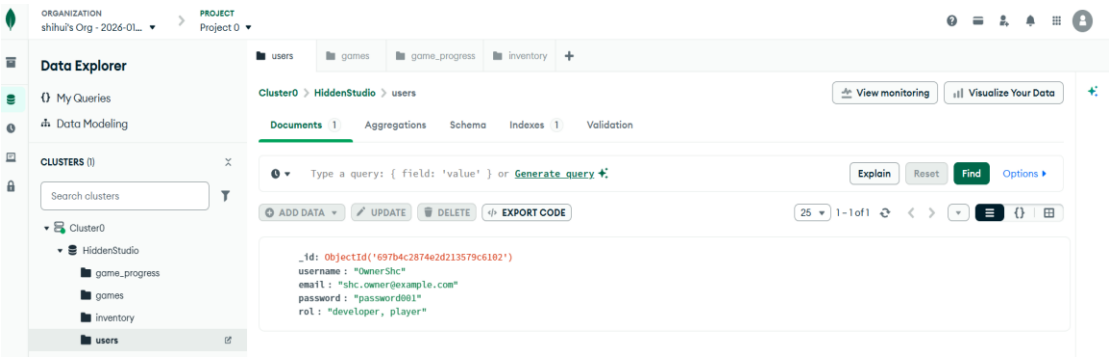
Por este motivo, se ha optado por una arquitectura monolítica en capas. Este enfoque permite una implementación más sencilla, una mejor comprensión global del sistema y un mantenimiento más directo, sin renunciar a una correcta separación de responsabilidades.

Además, la arquitectura elegida permite que, en un futuro, el sistema pueda evolucionar hacia un modelo más distribuido si el crecimiento del proyecto lo requiere.

BASE DE DATOS

El sistema utilizado es una base de datos MongoDB, un sistema de base de datos NoSQL orientado a documentos. Esta elección se debe a su flexibilidad a la hora de representar estructuras de datos, su fácil integración con Spring Boot y su adecuación para proyectos de desarrollo web con requisitos cambiantes.

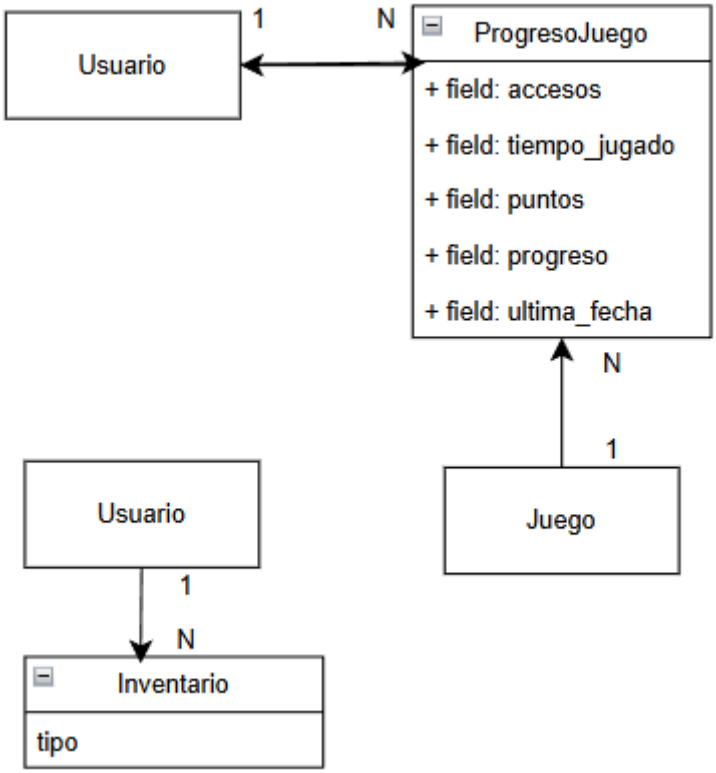
Así es cómo se ve gráficamente la base de datos en el MongoDB Cloud:



Las entidades principales de la base de datos son:

- Usuario (id, username, email, password, role): representa a los usuarios registrados en la plataforma.
- Juego (id, titulo, descripción, genero, urlWebGL, urlDescarga): representa los videojuegos desarrollados por Hidden Studio y disponibles en la plataforma.
- ProgresoJuego (id, usuario_id, juego_id, accesos, tiempo_jugado, puntos, progreso, ultima_fecha): permite almacenar de forma persistente estadísticas como el número de accesos, el tiempo total jugado, los puntos acumulados y el progreso del usuario en cada videojuego.
- Inventario (id, usuario_id, juego_id, tipo): permite gestionar el contenido al que un usuario tiene acceso dentro de la plataforma, como juegos desbloqueados o funcionalidades premium.

La relación entre Usuario, ProgresoJuego, Juego e Inventario se construye como en el diagrama de relaciones:



En MongoDB, estas relaciones se gestionan mediante referencias entre documentos, utilizando identificadores (usuario_id, juego_id) para mantener la coherencia de los datos y garantizar la integridad de las relaciones.

Este esquema representa la relación entre las principales entidades del sistema. La entidad intermedia ProgresoJuego permite almacenar información persistente del progreso del usuario en cada juego, incluyendo accesos realizados, tiempo jugado, puntos acumulados y última fecha de acceso.

OPERACIONES CRUD Y CONSULTAS

El sistema también implementa operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre las principales entidades:

- Usuarios: registro, modificación de datos, actualización de contraseñas y eliminación de cuentas.

- Juegos: alta, actualización de información, modificación de recursos (URL WebGL, descargas) y eliminación.
- ProgresoJuego: creación del registro al primer acceso del usuario a un juego, actualización de accesos, tiempo jugado, puntos y progreso, y lectura de estadísticas.
- Inventario: no se mostrará en la web, pero tiene como función de desbloqueo en el futuro para la expansión de funcionamientos de la aplicación como dar de alta, actualización y eliminación de ítems o logros asociados a cada usuario.

El sistema implementa consultas personalizadas que permiten obtener información más compleja que la proporcionada por el CRUD básico. Estas consultas facilitan el seguimiento de la actividad de los usuarios y la generación de estadísticas sobre su interacción con los juegos.

Ejemplos implementados:

- Obtener todos los juegos jugados por un usuario: permite conocer qué juegos ha iniciado cada usuario, junto con su progreso y puntos acumulados.
- Calcular el número total de accesos y tiempo jugado por un usuario: permite sumar todas las sesiones de un usuario en los distintos juegos, obteniendo información sobre su actividad y dedicación a la plataforma.

Hemos utilizado DTOs para transferir información entre el backend y el frontend de manera segura y eficiente. Los DTOs permiten enviar únicamente los datos necesarios, evitando exponer información sensible como contraseñas y desacoplando la estructura de la base de datos de la capa de presentación.

Ejemplos de DTOs en el proyecto:

- UserDTO: Contiene información básica del usuario como id, username, email y role, sin incluir la contraseña.
- GameProgressDTO: Contiene usuarioId, juegoId, progreso, accesos, tiempo jugado y puntos acumulados.

```java
public record UserDTO(  8 usages
        String id,  no usages
        String username,  no usages
        String email,  no usages
        List<String> roles,  no usages
        List<User> friends  no usages
) {
    public static UserDTO getUserDTO(User user){  1 usage
        return new UserDTO(
                user.getId(),
                user.getUsername(),
                user.getEmail(),
                user.getRole(),
                user.getFriends()
        );
    }
}
```

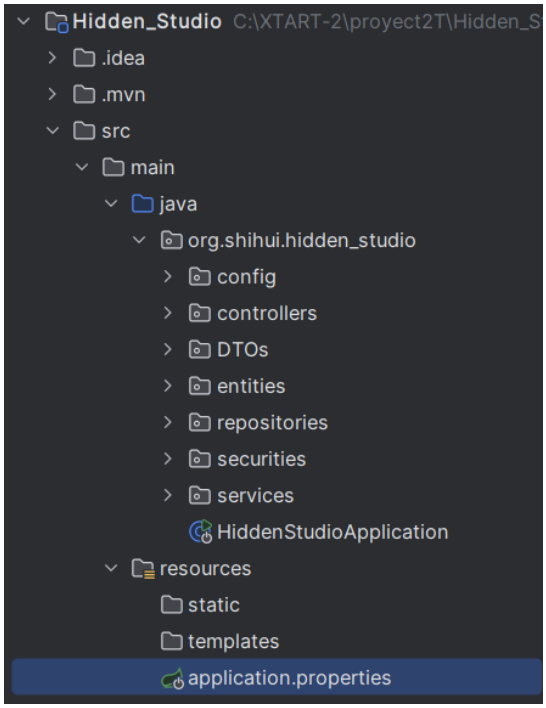
El uso de DTOs garantiza que el frontend reciba únicamente la información relevante y mantiene la seguridad y la coherencia de la aplicación.

Acceso a la base de datos

La base de datos debe estar encendida para que la aplicación funcione tanto para el backend como el frontend, se ha mandado un correo al Outlook para poder crear usuario en MongoDB y acceder a la base de datos, cuando se conecte el servidor (cluster0) podrá lanzar la aplicación java, para Angular será necesario tener lanzado estos dos proyectos.

PSP & ACCESO A DATOS (BACKEND)

El backend desarrollado se divide en varias capas para tener una estructura limpia y fácil de visualizar y manejar:



En toda la estructura del código se ha implementado un sistema de autenticación y autorización basado en OAuth2 utilizando Spring Security como Resource Server y JWT como formato de token.

La aplicación actúa como proveedor de autorización emitiendo tokens JWT tras la autenticación del usuario y del que el frontend Angular desarrollado consumirá los servicios REST enviando el token en la cabecera Authorization (Bearer Token).

Empezando por la capa de controller, se encarga de recibir las peticiones HTTP del frontend y exponer los endpoints REST del sistema. Gestiona las rutas, los parámetros de entrada y las respuestas, devolviendo los códigos HTTP adecuados según el resultado de cada operación. Además, esta capa se mantiene libre de lógica de negocio, delegando dicha responsabilidad en la capa de servicios para cumplir con el principio de separación de responsabilidades:

Entre todos los controladores creados destaca el AuthController, sobre todo porque no pertenece a los controladores normales que manejan las entidades de dominio del sistema.

Este componente es el encargado de gestionar el ciclo de vida de la autenticación y el registro de usuarios en la aplicación, actuando como la puerta de enlace entre las peticiones externas (Frontend en Angular) y la lógica de seguridad del servidor:

```
@RestController
@RequestMapping("/auth")
@CrossOrigin(origins = "http://localhost:4200")
public class AuthController {

    private final UserService userService;  2 usages
    private final AuthenticationManager authenticationManager;  2 usages
    private final JwtService jwtService;  2 usages

    public AuthController(UserService userService, AuthenticationManager authenticationManager, JwtService jwtService) {
        this.userService = userService;
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
    }

    @PostMapping("/register")
    public ResponseEntity<User> register(@RequestBody User user) {
        return ResponseEntity.ok(userService.crearUsuario(user));
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(@RequestBody LoginRequest loginRequest) {
        Authentication authentication = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                        loginRequest.username(),
                        loginRequest.password()
                )
        );
```

La capa service contiene la lógica de negocio de la aplicación y se encarga de procesar la información recibida desde los controladores, aplicar reglas de negocio y coordinar el acceso a los datos mediante la capa repository:

Podemos ver un ejemplo de servicio de juego con las operaciones CRUD básicas, donde se implementan las funciones de creación, lectura, actualización y eliminación, delegando el acceso en la base de datos desde su repositorio correspondiente:

```
@Service  3 usages
public class GameService {

    private final GameRepository gameRepository;  6 usages

    public GameService(GameRepository gameRepository) {
        this.gameRepository = gameRepository;
    }

    public List<Game> listarTodo() {  1 usage
        return gameRepository.findAll();
    }

    public Optional<Game> getGameById(String id) {  1 usage
        return gameRepository.findById(id);
    }

    public Game crearGame(Game game) {  1 usage
        return gameRepository.save(game);
    }

    public Game actualizarGame(Game game) {  1 usage
        return gameRepository.save(game);
    }

    public void eliminarGame(String id) {  1 usage
        gameRepository.deleteById(id);
    }

}
```

En los servicios destaca la variedad de funciones posibles a realizar relacionadas con el usuario y el progreso de juego. Para ello, se han creado consultas personalizadas con su propia lógica de negocio, definidas en el repositorio y utilizadas posteriormente en el controlador.

Podemos ver un ejemplo de nuevos método de progreso de juego que permite consultar el tiempo total que el jugador ha estado en un juego o las veces de acceso realizados a tal juego:

```
    //personalizadas

    public Double getTiempoTotalUsuario(String usuarioId) {  1 usage
        return gameProgressRepository.findByUsuarioId(usuarioId).stream()  Stream<GameProgress>
                .mapToDouble(GameProgress::getTiempoJugado)  DoubleStream
                .sum();
    }

    public int getAccesosTotalesUsuario(String usuarioId) {  1 usage
        return gameProgressRepository.findByUsuarioId(usuarioId).stream()  Stream<GameProgress>
                .mapToInt(GameProgress::getAccesos)  IntStream
                .sum();
    }
```

Al igual que para funciones automáticas, como el objetivo de desarrollar una función que calcule automáticamente el tiempo que ha estado cada jugador en un juego y por tanto hemos creado métodos tal que una vez iniciado un juego desde el frontend de angular detecte el registro de acceso y empiece a calcular el tiempo jugado del usuario de tal juego y los almacene como información:

```
//almacen temporal: user&gameId - hora_inicio
private final Map<String, LocalDateTime> sessionCache = new ConcurrentHashMap<>();  3 usages

//para sesion, inicio
public void iniciarSesion(String usuarioId, String juegoId) {  1 usage
    sessionCache.put(usuarioId + "-" + juegoId, LocalDateTime.now());
}
//fin
public GameProgress registrarAcceso(String usuarioId, String juegoId, int progresoSesion, int puntosSesion) {  1
    String key = usuarioId + "-" + juegoId;
    LocalDateTime inicio = sessionCache.get(key);

    double tiempoCalculado = 0.0;
    if (inicio != null) {
        tiempoCalculado = Duration.between(inicio, LocalDateTime.now()).getSeconds();
        sessionCache.remove(key);
    }
    GameProgress gameProgress = gameProgressRepository.findByUsuarioIdAndJuegoId(usuarioId, juegoId)
            .orElseGet(() -> {
                GameProgress nuevo = new GameProgress();
                nuevo.setUsuarioId(usuarioId);
                nuevo.setJuegoId(juegoId);
                nuevo.setAccesos(0);
                nuevo.setTiempoJugado(0.0);
                return nuevo;
            });
```

Es destacable que, como MongoDB no dispone de un tipo nativo equivalente a LocalDateTime, los valores de fecha y hora se almacenan internamente en formatos compatibles.

En el código de la aplicación se utiliza LocalDateTime y se ha configurado una conversión específica para MongoDB con el objetivo de guardar automáticamente la fecha de la última actualización de los registros:

(conversores MongoDB)

```
@Configuration
public class MongoConfig {

    @Bean
    public MongoCustomConversions customConversions() {
        return new MongoCustomConversions(List.of(
                Jsr310Converters.LocalDateTimeToDateConverter.INSTANCE,
                Jsr310Converters.DateToLocalDateTimeConverter.INSTANCE
        ));
    }

}
```

Gracias a la clase de apoyo “Jsr310Converters” que forma parte del ecosistema de Spring Data Core, nos ahorramos la necesidad de desarrollar por sí mismo las clases de conversión de LocalDateTime a Date y del revés. Nos ha permitido que los módulos de Spring Data, como Spring Data MongoDB o JPA, se transformen automáticamente tipos modernos como LocalDate o Instant en formatos compatibles con bases de datos o tipos legados como java.util.Date.

En los services creados se sitúa un servicio que forma parte de las seguridades empleadas en la aplicación, MyUserDetailsService, que define qué usuarios pueden autenticarse y cómo se realiza la carga de datos del usuario desde la base de datos:

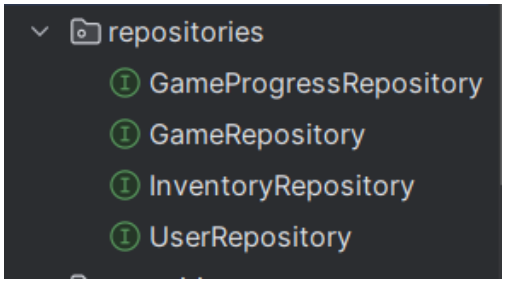
```java
@Service
public class MyUserDetailsService implements UserDetailsService {

    private final UserService userService;  2 usages

    public MyUserDetailsService(UserService userService) {
        this.userService = userService;
    }

    @Override  no usages
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userService.findByUsername(username);

        //convertir los roles en autoridades para Spring
        List<SimpleGrantedAuthority> authorities = user.getRole().stream() Stream<String>
                .map(SimpleGrantedAuthority::new) Stream<SimpleGrantedAuthority>
                .toList();

        return new org.springframework.security.core.userdetails.User(
                user.getUsername(),
                user.getPassword(),
                authorities
        );
```

Y será llamado en la configuración de security para que lo use en la identificación del usuario en el logueo de la página web, el sistema buscará en la base de datos MongoDB los datos de identificación correspondientes para la comparación de credenciales introducidas por el usuario, permitiendo o denegando el acceso según corresponda:

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final MyUserDetailsService userDetailsService;  2 usages
    private final BCryptPasswordEncoder passwordEncoder;  2 usages

    public SecurityConfig(MyUserDetailsService userDetailsService, BCryptPasswordEncoder passwordEncoder) {
        this.userDetailsService = userDetailsService;
        this.passwordEncoder = passwordEncoder;
    }

    @Bean
    public AuthenticationManager authenticationManager(HttpSecurity http) throws Exception {
        AuthenticationManagerBuilder authenticationManagerBuilder =
                http.getSharedObject(AuthenticationManagerBuilder.class);

        authenticationManagerBuilder
                .userDetailsService(userDetailsService)
                .passwordEncoder(passwordEncoder);
```

La capa repository utiliza Spring Data MongoDB para gestionar el acceso a la base de datos.

Permite realizar operaciones CRUD y consultas personalizadas sobre las entidades, abstrayendo el acceso a los datos del resto de la aplicación.



Podemos ver un ejemplo con el repositorio de proceso de juego, como vemos, de las consultas predefinidas y disponibles de MongoDB que queremos usar en nuestro código las hemos definido en su repositorio:

```
@Repository  3 usages
public interface GameProgressRepository extends MongoRepository<GameProgress,String> {

    Optional<GameProgress> findByUsuarioIdAndJuegoId(String usuarioId, String juegoId);  1 usage
    List<GameProgress> findByUsuarioId(String usuarioId);  3 usages
    List<GameProgress> findByJuegoId(String juegoId);  no usages

}
```

Las entidades representan el modelo de datos del sistema y están directamente relacionadas con la base de datos.

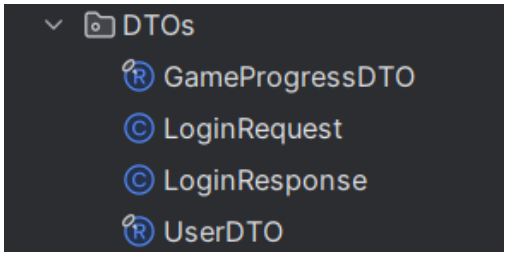
Cada entidad define los atributos y relaciones necesarias para almacenar la información persistente de la aplicación:

Podemos ver que hemos llamado todos los campos del documento de Usuario y también aclarado de los nombres en el que se encuentran en la base de datos de MongoDB:

```java
@Document(collection = "users")  43 usages
public class User {

    @Id
    @Field(name = "_id")
    private String id;

    private String email;  3 usages
    private String password;  3 usages
    private List<String> roles;  3 usages
    private String username;  3 usages

    private List<User> friends;  3 usages

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getEmail() {  1 usage
```

Los DTOs (Data Transfer Objects) se utilizan para transferir datos entre el backend y el frontend sin exponer directamente las entidades del sistema.

Permiten enviar únicamente la información necesaria y mejorar la seguridad y el desacoplamiento entre capas, a parte de los dos modelos de las clases, tendremos otras dos para que nuestro frontend y Java hablen el mismo idioma durante el proceso de login:



Podemos ver que estos dos objetos de login tienen la responsabilidad de que Angular y Java hablen el mismo idioma, por ejemplo, para recibir el JSON con usuario y contraseña:

```java
public record LoginRequest(  2 usages
        String username,  1 usage
        String password  1 usage
) {}
```

De los DTOs utilizados para las entidades, podemos ver como ejemplo el DTO de usuario, implementado mediante un *record*.

Este diseño permite exponer únicamente la información filtrada del usuario, evitando enviar datos sensibles como la contraseña, garantizando así una mayor seguridad en la comunicación con el frontend:

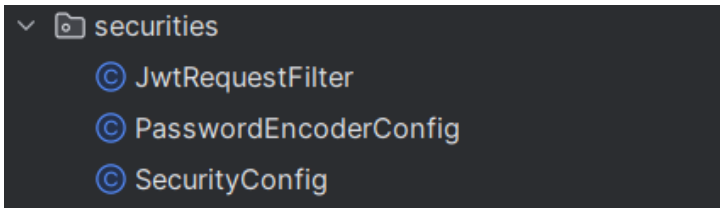
```
public record UserDTO(  8 usages
        String id,  no usages
        String username,  no usages
        String email,  no usages
        List<String> roles,  no usages
        List<User> friends  no usages
) {
    public static UserDTO getUserDTO(User user){  1 usage
        return new UserDTO(
                user.getId(),
                user.getUsername(),
                user.getEmail(),
                user.getRole(),
                user.getFriends()
        );
    }
}
```

Como caso de uso, tenemos por ejemplo en el servicio para usuario para la consulta de mostrar todos los usuarios, aunque esta consulta solo será realizable por los usuarios que sean administradores como podemos ver anteriormente en los controladores:

```
    }

    public List<UserDTO> listarTodoUser() {  1 usage
        return userRepository.findAll().stream()  Stream<User>
                .map(UserDTO::getUserDTO)  Stream<UserDTO>
                .toList();
    }

    public Optional<User> getUserById(String id) {  no usages
```

La capa security gestiona la autenticación y autorización del sistema mediante una arquitectura basada en Spring Security, JWT y el uso de DTOs (Data Transfer Objects).

Esta capa define los filtros de seguridad, la política IF_REQUIERED de autenticación y las reglas de acceso a los endpoints protegidos de la API:



Es en esta capa es donde hemos determinado el funcionamiento del login y las configuraciones de seguridad relativas a las peticiones que se reciben desde el puerto 4200 correspondiente a Angular, configurando correctamente CORS para permitir la comunicación entre frontend y backend:

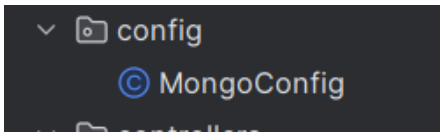
```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http, JwtRequestFilter jwtRequestFilter) throws Exception {
    http
            .cors(withDefaults())
            .csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())

            .sessionManagement( SessionManagementConfigurer<HttpSecurity> session -> session
                    .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
            )

            .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                    .requestMatchers("/auth/**", "/login", "/error").permitAll()
                    .anyRequest().authenticated()
            )

            .formLogin( FormLoginConfigurer<HttpSecurity> form -> form
                    .loginPage("/login")
                    .permitAll()
                    .defaultSuccessUrl( defaultSuccessUrl: "/me/progress", alwaysUse: true)
```

También se encarga de validar las credenciales del usuario según los datos almacenados en MongoDB, garantizando que solo los usuarios autenticados y con los roles adecuados (verificados mediante @PreAuthorize) puedan acceder a los recursos protegidos de la API

Podemos ver un ejemplo en el controlador de usuarios, dependiendo de si es rol administrador podrá o no realizar la siguiente acción:

```
public UserController(UserService userService) { this.userService = userService; }

//segun permiso
@GetMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<List<UserDTO>> getAllUsers() { return ResponseEntity.ok(userService.listarTodoUser()); }

@GetMapping("/me")
public ResponseEntity<User> getMyProfile(Authentication authentication) {
```

La capa de configuración lo usaremos para desarrollar algunas clases que necesitemos para el código, como hemos mencionado antes, el MongoConfig, que en este proyecto será la única:



En ella hemos configurado para que se transformen los datos de fecha y tiempo generados en la aplicación en un formato entendible para la base de datos MongoDB y que los almacene correctamente:

```java
@Configuration
public class MongoConfig {

    @Bean
    public MongoCustomConversions customConversions() {
        return new MongoCustomConversions(List.of(
                Jsr310Converters.LocalDateTimeToDateConverter.INSTANCE,
                Jsr310Converters.DateToLocalDateTimeConverter.INSTANCE
        ));
    }

}
```

CRUD

Dentro del código backend desarrollado, encontramos que se implementa el CRUD completo para las principales entidades del sistema, permitiendo la creación, lectura, actualización y eliminación de usuarios, juegos y registros de progreso.

A parte de la de juegos que hemos mencionado en la parte de servicios, otro ejemplo que se vea claro puede ser el de inventario:

```java
    public List<Inventory> listarTodo() { return inventoryRepository.findAll(); }

    public Optional<Inventory> getInventoryById(String id) {  2 usages
        return inventoryRepository.findById(id);
    }

    public List<Inventory> getInventoryByUserId(String userId) { return inventoryRepository.findByUsuarioId(userId); }

    public Inventory crearInventory(Inventory inventory) { return inventoryRepository.save(inventory); }

    public Inventory actualizarInventory(Inventory inventory) { return inventoryRepository.save(inventory); }

    public void eliminarInventario(String id) { inventoryRepository.deleteById(id); }

}
```

Además del CRUD básico, se implementan consultas personalizadas para obtener información agregada y estadística, como el número de accesos a un juego, el tiempo total jugado por usuario en el progreso de juego o también el método de añadir amigos en servicio o controlador de usuarios, estas funciones aún no se usarán todos en nuestra aplicación general, pero podrán ser muy útiles en futuros avances:

```java
//personalizado
public User addFriend(String userId, String friendId) {  1 usage
    User friend = userRepository.findById(friendId)
            .orElseThrow(() -> new RuntimeException("no existe ese usuario"));
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new RuntimeException("no existe ese usuario"));

    if (user.getFriends().stream().noneMatch( User u -> u.getId().equals(friendId))) {
        user.getFriends().add(friend);
        friend.getFriends().add(user);

        userRepository.save(friend);
        return userRepository.save(user);
    }

    return user;
}

public User removeFriend(String userId, String friendId) {  1 usage
```

Spring Security, JWT y DTOs

La autenticación se realiza mediante el motor de Spring Security, validando las credenciales del usuario contra los registros almacenados en MongoDB.

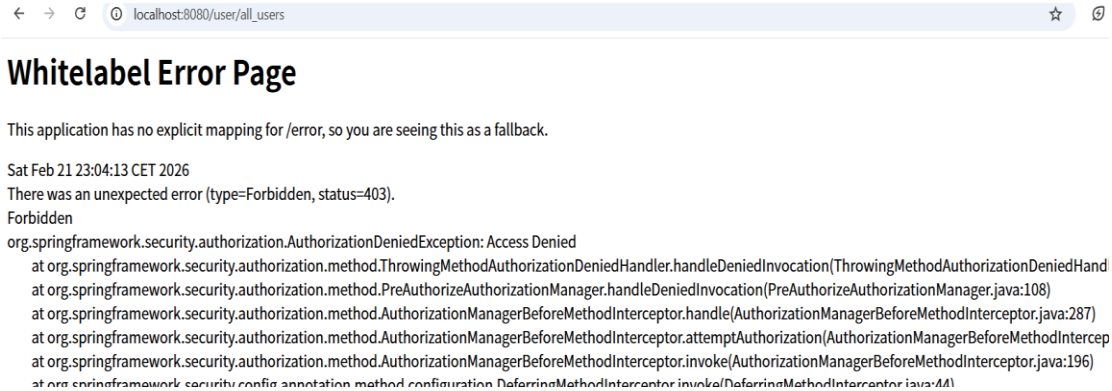
Tras un inicio de sesión exitoso, el servidor genera una respuesta estructurada mediante un LoginResponse (DTO). Este objeto no solo devuelve el nombre de usuario y su lista de roles, sino que incorpora un token JWT (JSON Web Token) generado dinámicamente sobre todo para el funcionamiento del frontend.

Este token actúa como una credencial efímera y segura que el cliente (Angular) almacena y adjunta en el encabezado Authorization de cada petición HTTP. En el backend, el filtro de seguridad personalizado (JwtRequestFilter) intercepta estas peticiones, valida la firma del token y establece el contexto de autenticación de forma stateless (sin estado).

Este flujo garantiza la seguridad de la API mediante un control de acceso basado en roles (RBAC), permitiendo que el servidor identifique al usuario y proteja los endpoints mediante anotaciones @PreAuthorize que admitimos este tipo de configuración gracias a la anotación de @EnableMethodSecurity en la configuración de seguridad. Al utilizar objetos de transferencia de datos (DTOs), se asegura que la información sensible, como las contraseñas encriptadas, nunca sea expuesta en las peticiones hacia el cliente (Angular).

Podemos ver un ejemplo con usuarios de distintos permisos, probaremos con la dirección de ver todos los usuarios que hemos puesto como necesidad de acceso con permiso administrador, entramos primero con un usuario normal:

Y nos metemos en la dirección de ver todos los usuarios que tenemos declarado en el código:



Vemos en el error que nos niega el acceso con el usuario user002, ahora entremos con nuestro usuario con permiso total:

localhost:8080/user/all_users

Nuevo Chrom

)ar formato al texto


```
[
  {
    "id": "69830cfc6cab6670ed69c0b8",
    "username": "ginkgoFox",
    "email": null,
    "roles": [
      "ROLE_ADMIN",
      "ROLE_DEVELOPER",
      "ROLE_USER"
    ],
    "friends": null
  },
  {
    "id": "69846ca11f3e8bfd35bf1cd0",
    "username": "user002",
    "email": null,
    "roles": [
      "ROLE_USER"
    ],
    "friends": null
  }
]
```

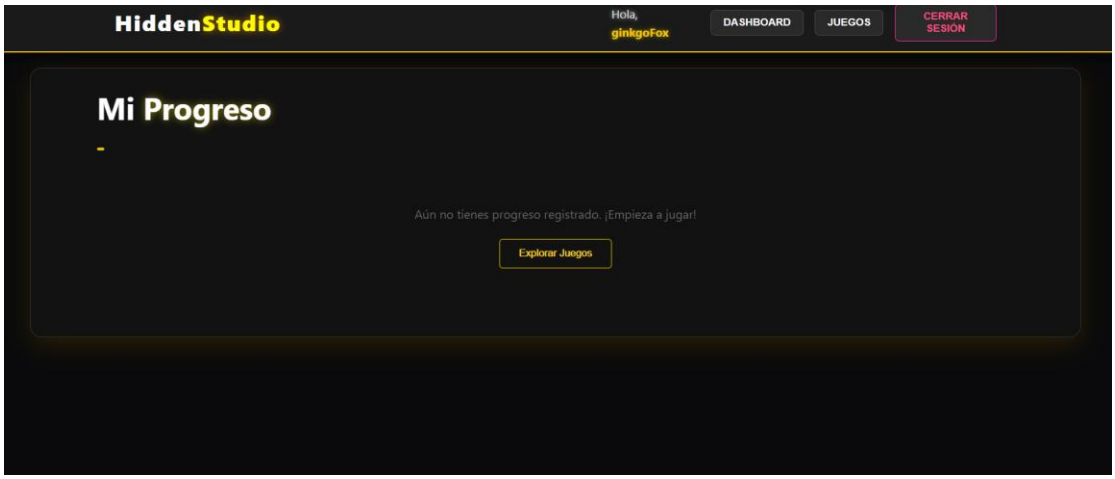

Podemos observar que la información se devuelve filtrada mediante el UserDTO, evitando exponer las contraseñas incluso en su forma codificada.

Gracias a este sistema de permisos se garantiza el principio de mínimos privilegios, proporcionando accesos diferenciados según el rol del usuario y asegurando la protección de los datos del sistema.

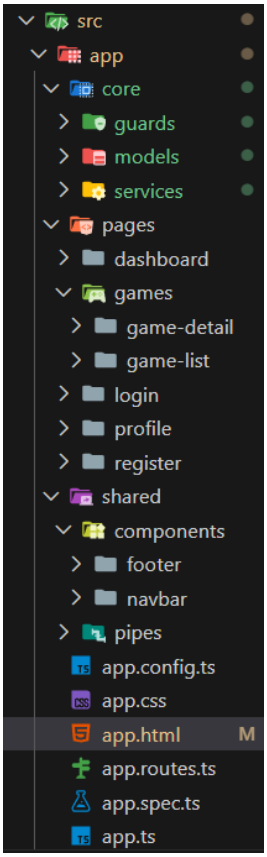
En resumen, el sistema cuenta con autenticación de usuarios mediante credenciales básicas. En una visión futura, implementar seguridad avanzada mediante JWT se considera como una buena mejora, ya que el proyecto se desarrolla como un proyecto con backend en entorno local y que tiene objetivo de demostrar una idea.

DESARROLLO DE INTERFACES – ANGULAR (FRONTEND)

El frontend de la aplicación está desarrollado utilizando Angular, siguiendo una arquitectura basada en componentes y servicios.



La aplicación funciona como una Single Page Application (SPA), permitiendo una navegación fluida sin recargas completas de página. La comunicación con el backend se realiza mediante servicios Angular que consumen la API REST desarrollada en Spring Boot, utilizando peticiones HTTP y autenticación basada en JWT.



Las rutas implementadas en el proyecto para organizar son:

- */login* - Inicio de sesión de usuarios
- */register* - Registro de nuevos usuarios
- */dashboard* - Panel principal del usuario
- */games* - Listado de juegos disponibles
- */games/:id* - Detalle de un juego concreto
- */profile* - Perfil del usuario

Algunas de ellas están protegidas mediante guardas de autenticación, evitando el acceso sin sesión iniciada.

Cada ruta de la aplicación se asocia a un componente específico, encargado de gestionar la vista y la lógica de presentación correspondiente.

Guards e interceptors

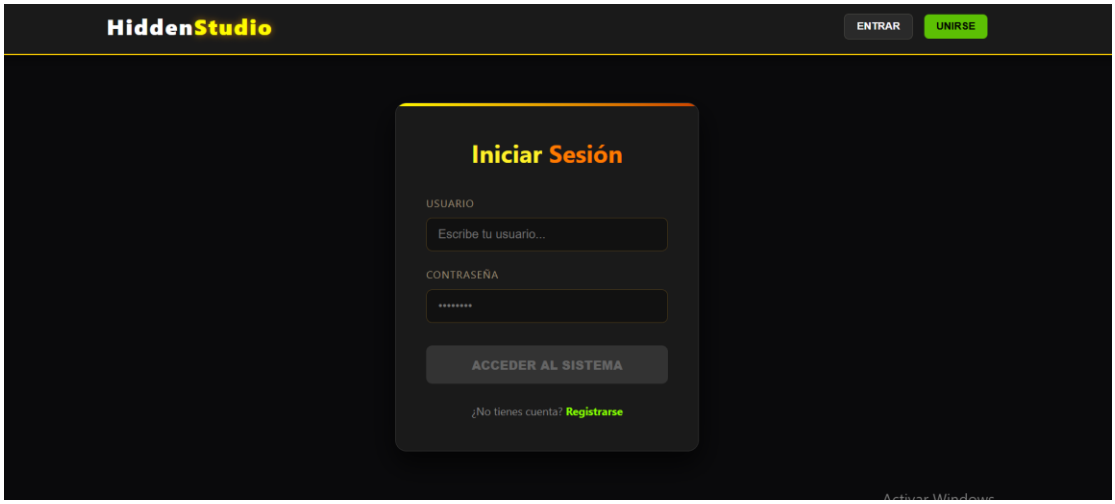
Para la conexión entre la base de datos y la web con el juego, hemos realizado la conexión del frontend de Angular con el backend de java montado con Spring Security de donde obtendrá los datos de la base de datos manteniendo la seguridad durante todo el proceso.

En guards se encuentra interfaces que actúan como "porteros" o puestos de control en la navegación. Su función principal es decidir si un usuario tiene permiso para dirigirse a una ruta específica o para salir de ella tales como administrador o usuario normal. En interceptors recibimos y mandamos el token en el header como autorización para autenticarnos y acceder a la página para ver contenidos.

En nuestro caso no hemos implementado totalmente la lógica de funciones según roles, pero en el futuro desarrollo y expansión de la web serán muy necesarios, aquí tenemos por ejemplo la interfaz clase de auth que identifica si el usuario está logueado:

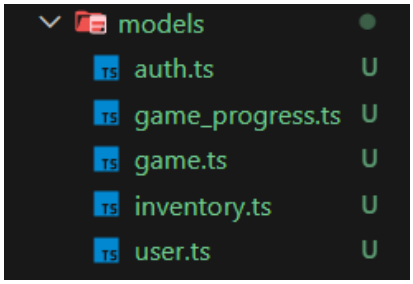
```
auth.ts U ×
Hidden_Studio > src > app > core > guards > auth.ts > ...
1  import { inject } from '@angular/core';
2  import { Router, CanActivateFn } from '@angular/router';
3  import { AuthService } from '../services/auth';
4
5  export const authGuard: CanActivateFn = (route, state) => {
6    const authService = inject(AuthService);
7    const router = inject(Router);
8
9    if (authService.currentUser()) {
10     return true;
11   }
12
13   router.navigate(['/login']);
14   return false;
15 };
```

Y tenemos la página de inicio de sesión así, detectará si el usuario introducido existe o no gracias a las funciones desarrolladas en el backend java conectado con la base de datos de MongoDB:



Models

Aquí tenemos la representación de los datos con los que vamos a trabajar:



Definimos la "forma" de los objetos para que el código sea más seguro, organizado y fácil de entender. Por ejemplo, en el de progreso de juego recibimos estos atributos desde el backend y que los usaremos en otras zonas como en servicios o mostrar datos en las páginas:

```
1  export interface GameProgressDTO {
2    juegoId: string;
3    accesos: number;
4    tiempoJugado: number;
5    progreso: number;
6    puntos: number;
7  }
```

Services

Tenemos aquí las clases especializadas cuya única misión es manejar la lógica de negocio y los datos, manteniéndolos separados de la interfaz de usuario.

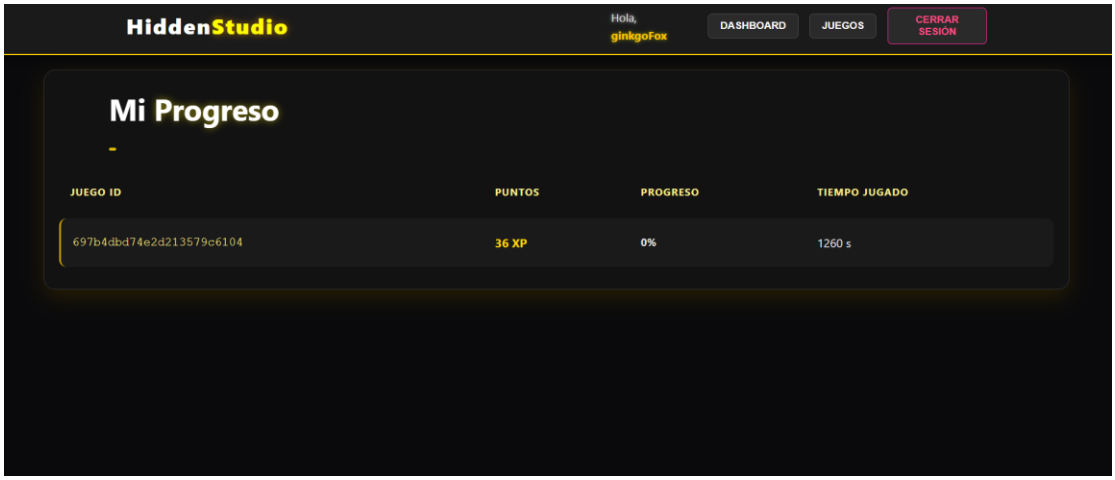
Encontramos, por ejemplo, métodos en el de autenticación, usados en el inicio de sesión o registro:

```
12  export class AuthService {
14    private router = inject(Router);
15
16    private readonly AUTH_URL = 'http://localhost:8080/auth';
17    private readonly USER_URL = 'http://localhost:8080/user';
18
19    private currentUserSignal = signal<LoginResponse | null>(this.getUserFromStorage());
20
21      currentUser = computed(() => this.currentUserSignal());
22
23      isAuthenticated = computed(() => !!this.currentUserSignal());
24
25      constructor() {}
26
27      //metodos de autenticacion
28      register(user: UserDTO): Observable<User> {
29          return this.http.post<User>(`${this.AUTH_URL}/register`, user);
30      }
31
32      login(credentials: LoginRequest): Observable<LoginResponse> {
33          return this.http.post<LoginResponse>(`${this.AUTH_URL}/login`, credentials).pipe(
34            tap((response: LoginResponse) => {
35              this.currentUserSignal.set(response);
36              localStorage.setItem('user', JSON.stringify(response));
37              localStorage.setItem('auth_token', response.token);
38            }),
39            catchError(error => {
40              console.error('Login error:', error);
41              return throwError(() => new Error('Login failed. Please check your credentials and try again.'));
42            })
43          );
```

Dashboard

Dentro de la carpeta de páginas, tenemos el de dashboard, página principal, que actúa como el panel principal del usuario, mostrando información relevante de forma resumida.

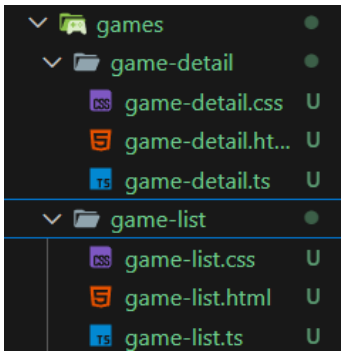
Los datos se obtienen mediante consultas al backend, permitiendo visualizar estadísticas como el número de juegos jugados, la puntuación total y el último acceso registrado, debido a la conexión con el backend, necesita dar siempre una segunda vez al botón de dashboard para mostrar los registros que se han guardado del usuario.



Esta aplicación diseñada permite la gestión y visualización de las principales entidades del sistema mediante componentes específicos, consumiendo los endpoints de la API mediante las peticiones REST lanzadas hacia el backend y es así cómo hemos conectado frontend y backend.

Games

En esta carpeta tenemos dividido para listar juegos disponibles, cómo listarlo, su descripción y también el cómo contener el juego jugable directamente en la web:



Contiene cada uno la lógica, orden de dónde situar cada apartado y los estilos *css* aplicados a la página, podemos ver así a la página que lista todos los juegos disponibles. Además, al final de esta página se encuentra también el espacio de las recomendaciones de otros juegos venidos de la API externa que hemos consumido y que lo veremos en otro apartado:

```html
game-list.html U ×
Hidden_Studio > src > app > pages > games > game-list > game-list.html > div.container > div.rawg-grid > a.game-card.external > div.game-card.external
<div class="container">
  <header class="catalog-header">
    <h2>Catálogo de Juegos</h2>
    <div class="divider"></div>
  </header>

  <div class="game-grid">
    @for (game of games; track $index) {
      <div class="game-card" [routerLink]="['/games', game.id]">
        <div class="card-image">
          <img [src]="game.urlPortada" [alt]="game.titulo" title="{{ game.titulo }}"/>
        </div>
        <div class="card-info">
          <h3>{{ game.titulo }}</h3>
          <span class="genre-tag">{{ game.genero }}</span>
        </div>
        <div class="card-overlay">
          <p class="game-description">{{ game.descripcion }}</p>
          <div class="card-actions">
            <button class="play-btn">VER JUEGO</button>
            @if (authService.hasRole('ADMIN')) {
              <button (click)="$event.stopPropagation(); eliminarJuego(game.id!)" class="btn-danger">ELIMINAR</button>
            }
          </div>
        </div>
      </div>
    } @empty {
      @for (i of [1,2]; track i) {
        <div class="game-card skeleton">
          <div class="card-image loading-bg"></div>
          <div class="card-info">
            <div class="skeleton-text title"></div>
```

En esta página, cuando el ratón se sitúa encima de la carta del juego se verá la descripción del juego guardado en la base de datos, al dar clic en cualquier zona de la carta puede direccionarle a la página que contiene juego incrustado para jugar o descargar (teniendo en cuenta que el progreso solo se guardará si se juega en la web):

Consumo de API externa

Además de la API propia del backend, el frontend consume una API externa pública de videojuegos (RAWG API) para obtener información adicional y en concreto, se integra la API RAWG para mostrar listados de juegos populares y datos adicionales, complementando los juegos propios almacenados en la plataforma.

Para consumir como API externa RAWG, en el servicio hemos generado un servicio para gestionar las llamadas externas:

```
@Injectable({
  providedIn: 'root'
})
export class RawgService {
  private http = inject(HttpClient);
  private readonly API_KEY = '57390d0ee257464eb7d9295544da3eba';
  private readonly BASE_URL = 'https://api.rawg.io/api';

  // Obtener juegos populares para la sección /games
  getPopularGames(): Observable<any[]> {
    return this.http.get<any>(`${this.BASE_URL}/games?key=${this.API_KEY}&page_size=10&ordering=-rating`)
      .pipe(map(response => response.results));
  }

  // Obtener noticias o lanzamientos recientes para el Dashboard
  getLatestReleases(): Observable<any[]> {
    return this.http.get<any>(`${this.BASE_URL}/games?key=${this.API_KEY}&dates=2025-01-01,2026-12-31&ordering=-added`)
      .pipe(map(response => response.results));
  }
}
```

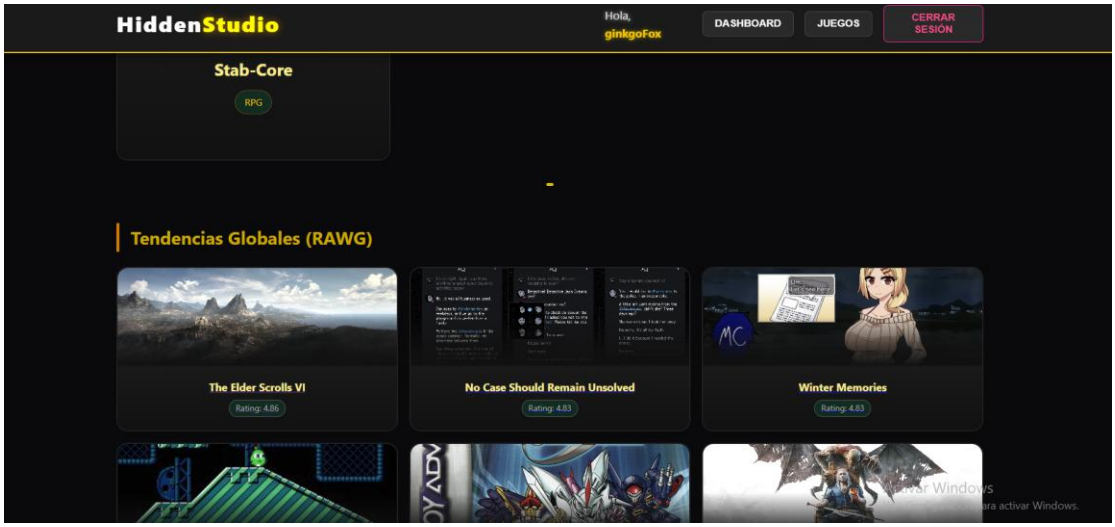
También hemos configurado el servicio de juegos para que realice las peticiones y que muestre nuestros juegos más los de RAWG:

```
export class GameListComponent implements OnInit {
  private gameService = inject(GameService);
  public authService = inject(AuthService);
  private cdr = inject(ChangeDetectorRef);
  private rawgService = inject(RawgService);

  games: Game[] = [];
  popularGames: any[] = [];

  ngOnInit(): void {
    this.gameService.getGames().subscribe({
      next: (data) => {
      console.log('Juegos recibidos:', data);
      this.games = [...data];
      this.cdr.detectChanges();
      },
      error: (err) => console.error('Error cargando juegos', err)
    });
    this.rawgService.getPopularGames().subscribe({
      next: (data) => {
        this.popularGames = data;
        this.cdr.detectChanges();
      },
      error: (err) => console.error('Error cargando juegos populares', err)
    });
  }
```

En el html también tenemos una sección para esos juegos:

```
        </div>
      }
    }
  </div>

  <div class="divider"></div>

  <h2 class="rawg-title">Tendencias Globales (RAWG)</h2>

  <div class="rawg-grid"  >
    @for (externalGame of popularGames; track externalGame.id) {
      <div class="game-card external">
        <div class="card-image">
          <img [src]="externalGame.background_image" [alt]="externalGame.name" title="{{ externalGame.name }}"/>
        </div>
        <div class="card-info">
          <h3>{{ externalGame.name }}</h3>
          <span class="genre-tag">Rating: {{ externalGame.rating }}</span>
        </div>
      </div>
    }
```

Esta integración permite enriquecer el contenido mostrado en la plataforma sin necesidad de almacenar esta información en la base de datos interna. Recordando que hemos mencionado antes, lo podemos encontrar en la web en la página de la lista de juegos:



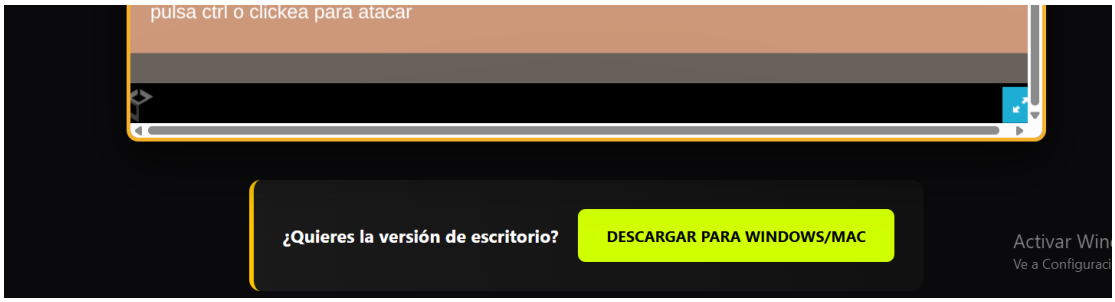
Hemos configurado también para que sea posible acceder a la página de información del juego en la API:

Todo esto forma parte del frontend desarrollado con Angular que proporciona una interfaz clara y funcional, integrándose con el backend mediante servicios REST y cumpliendo los requisitos de navegación, gestión de entidades y consumo de APIs externas del proyecto en sí.

PMDM – VIDEOJUEGO: STAB-CORE (UNITY)

El primer juego de la plataforma será un juego 3D llamado Stab-Core de tipo RPG, es diseñado con Unity y se ha creado desde ella modelos de personajes, enemigos, controladores y generadores junto a sus scripts que definen acciones y reacciones a las circunstancias que puede pasar.

El juego está disponible para jugarse directamente desde la web desarrollada en Angular, o bien puede descargarse el ejecutable, situado debajo del juego en la web, para jugarlo en el ordenador si el usuario lo prefiere, pero no se guardará su historial de mayor puntuación que, de momento, es la única jugabilidad.



Para integrar el juego dentro del sitio web, se ha utilizado el método de incrustación mediante un elemento *iframe*, lo que permite ejecutar la versión WebGL del proyecto de Unity directamente en el navegador, manteniendo la correcta visualización de los elementos gráficos y de la interfaz del juego.

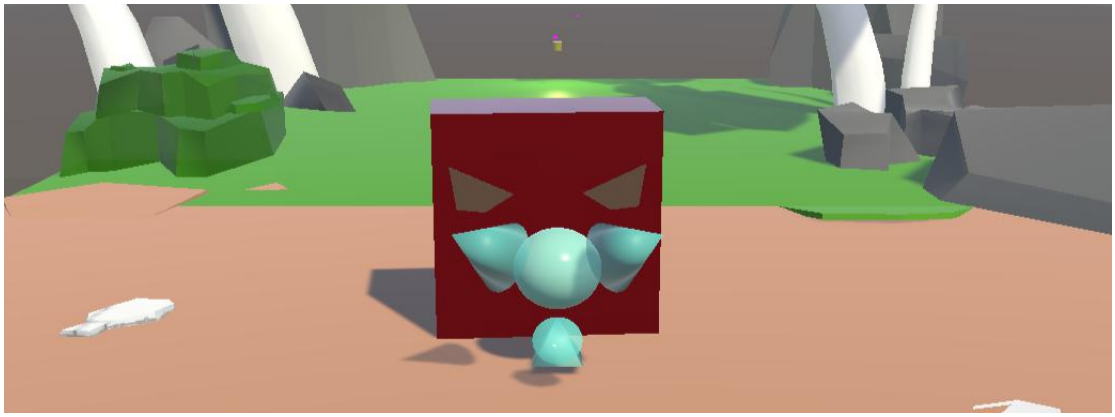
Así es cómo lo encontraremos en la página de juegos, donde en el futuro se podrá encontrar más juegos creados por la empresa:

Desde ella podemos acceder al juego y además se ha hecho una conexión con el backend para guardar registros del usuario en los juegos que haya jugado tanto como puntos máximos conseguidos en el juego o tiempo total jugado en ella:

JUEGO ID	PUNTOS	PROGRESO	TIEMPO JUGADO
697b4dbd74e2d213579c6104	36 XP	0%	355 s

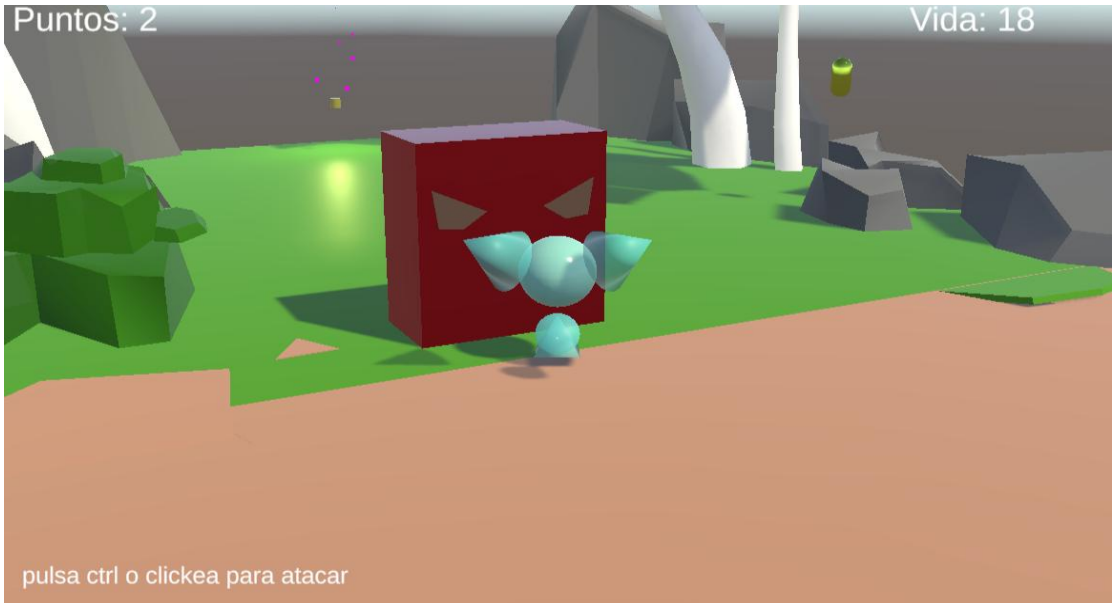
El juego está diseñado en 3D creando figuras con una herramienta llamado ProBuilder que venía ya integrado en Unity solo haciendo falta instalarlo desde el package manager del menú de Unity.

El objetivo del jugador es derrotar tantos enemigos que pueda (se acercan automáticamente a una distancia y sus ataques restan nuestra vida poco a poco) y ganar puntos para intentar llegar a su récord máximo, que lo tendrá guardado en la base de datos si juega directamente en la web, debe evitar caerse del plano y mantener la cantidad de vida, si cae o su vida llega a 0 se reseteará todo y el jugador deberá empezar de nuevo.

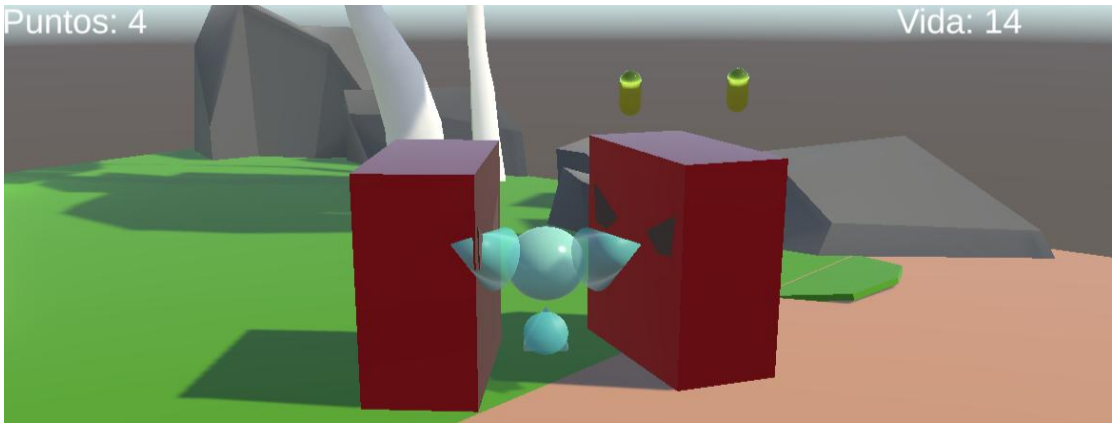


Al entrar encontrará el jugador un enemigo solamente para que pruebe los movimientos y ataque del personaje, si no lo derrota no podrá desbloquear el paso a la segunda zona y empezar el juego, en el juego puede saltar (espacio), atacar (ctrl o clic) y moverse (teclas de dirección) por todas las zonas disponibles, en el futuro podrá también guardar progreso y pasar a más zonas a desbloquear tipos de enemigos o ataques e incluso formas de juego. El usuario encontrará en la esquina inferior izquierdo una línea de instrucción de cómo atacar evitando la necesidad de leerse documentación del juego para que sea más intuitivo:

Al derrotarle empezará a generar enemigos infinitamente cada tres segundos, el usuario deberá tener cuidado de no caer del espacio y derrotar a los enemigos que ira saliendo, cada enemigo dará un punto que se mostrará en la esquina superior izquierda:

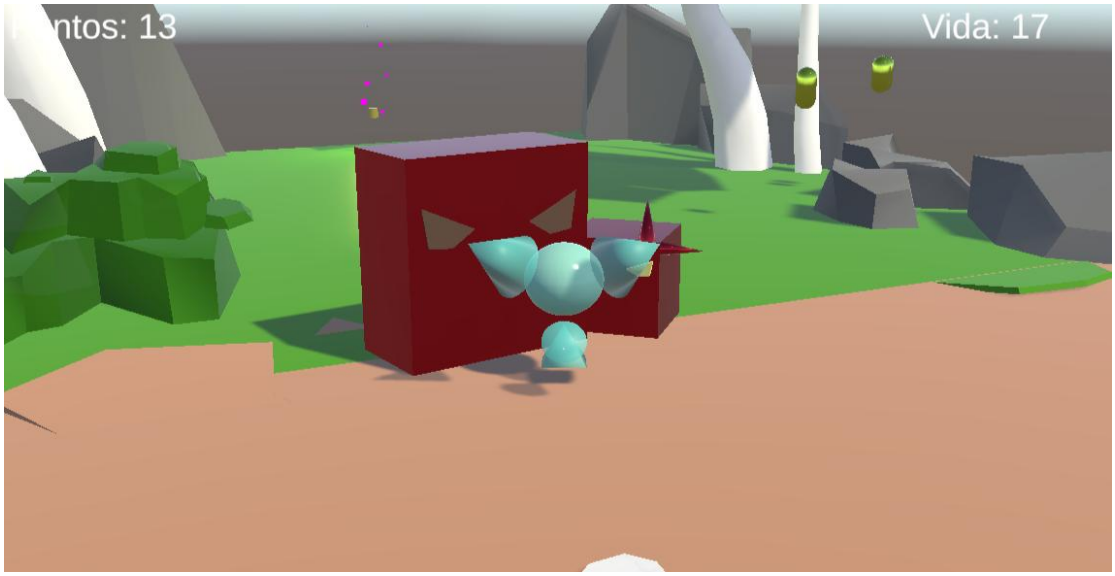


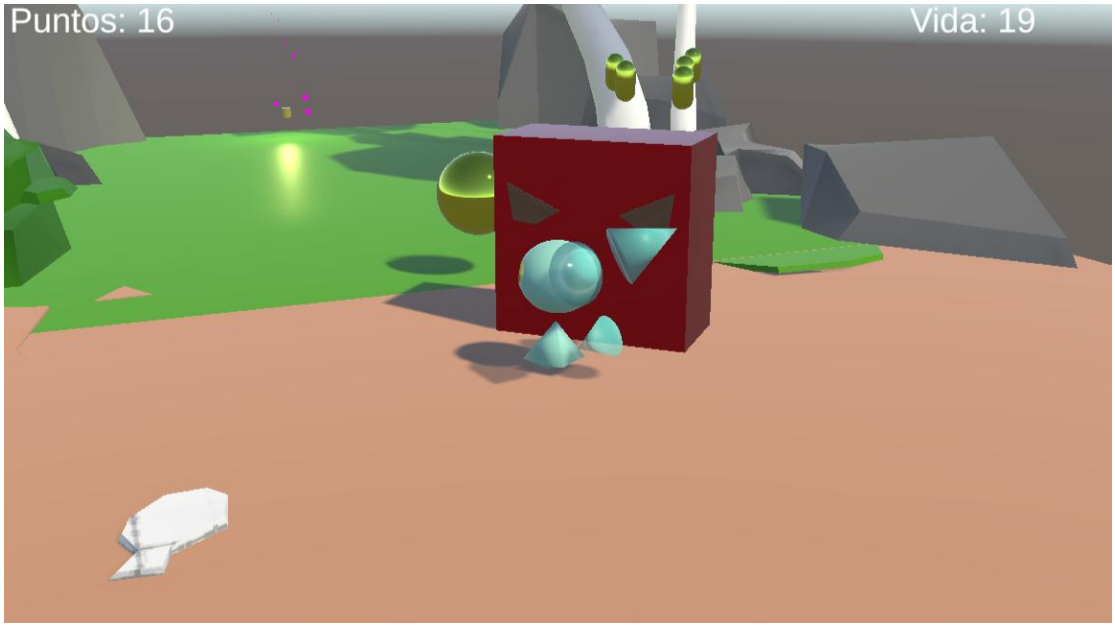
EL jugador no podrá entrar a la siguiente zona si no derrota al enemigo de prueba y tampoco empezará a generar cápsulas de recuperar vidas:



El jugador deberá saltar con la tecla de espacio para coger las cápsulas, cada capsula recupera 5 de vida, pero como hay un máximo de 20, el jugador no podrá tener más vida de lo limitado.

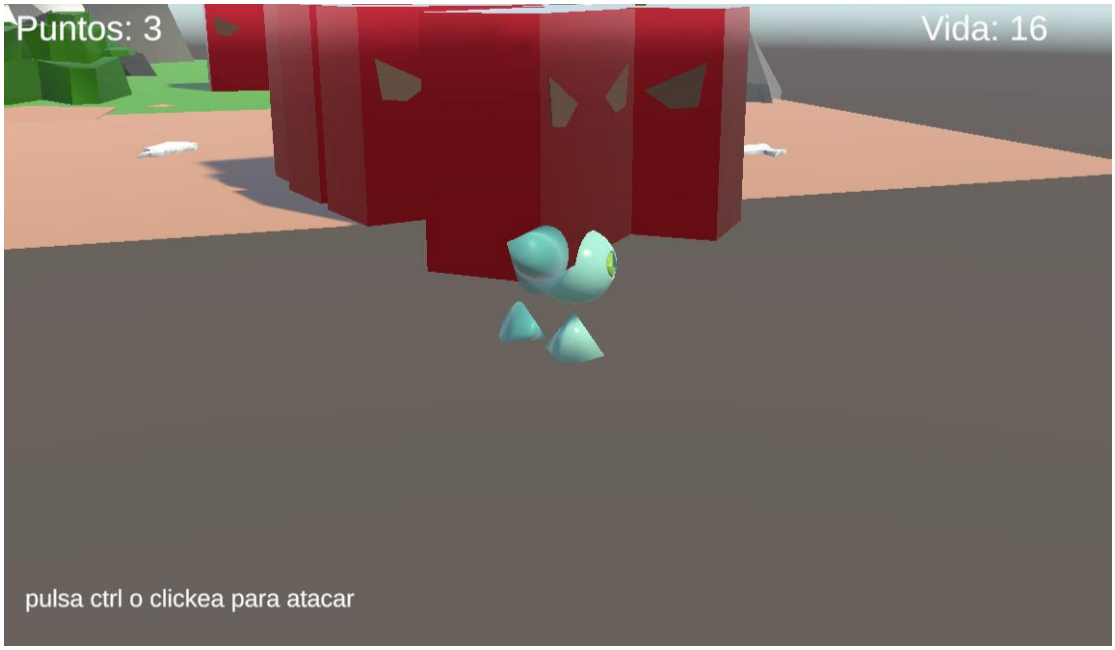
Una vez que consigue llegar a los 12 puntos se generará el primer enemigo boss, al derrotar el primer boss se generará una bola que cuando el jugador lo recoja desbloqueará el ataque especial, este ataque se dispara pulsando la tecla ‘p’ y hará más daño que la normal:





Después de que el primer boss haya generado, empezará a salir enemigos boss cada 15 segundos, estos enemigos hacen más daño que los enemigos normales, aunque ataca más lento y su zona de detección del jugador es más corto que los otros. Así que, el jugador en caso de tener muy baja vida puede alejarse del boss para evitar su ataque y recuperar un poco de vida.

Si se acumulan muchos enemigos te pueden empujar hasta que te caigas del plano, se reseteará todo y deberá empezar de nuevo:



Las funciones definidas en el script del juego mandarán los puntos al frontend y este al backend, ahí es donde calculará si es mayor o menor que los puntos registrados y actualizará en la base de datos los puntos máximos conseguidos en caso de que sea mayor.

El juego sigue aún en el proceso de desarrollo y diseño, por ser una idea algo complicada en desarrollar por el tiempo limitado, en un futuro llegará a crecer a más para diversificar la forma de juego, conformar los gustos de los distintos usuarios y expandir ideas únicas de diseño de juegos.

INTEGRACIÓN WEB + UNITY

La integración entre la plataforma web y los videojuegos desarrollados con Unity se ha diseñado teniendo en cuenta la accesibilidad y facilidad de uso permitiendo tanto el juego directo desde el navegador como la descarga del ejecutable en PC.

Para ello, se han definido diferentes modalidades de integración que permiten a los usuarios acceder a los juegos desde la plataforma web de forma segura y centralizada, manteniendo siempre la asociación con la cuenta del usuario autenticado en la plataforma.

Se ha realizado una integración mediante WebGL, lo que permite que los videojuegos desarrollados con Unity puedan exportarse en formato WebGL y ejecutarse directamente en el navegador web sin necesidad de instalaciones adicionales.

Esta modalidad permite una experiencia inmediata de juego, accesible desde cualquier dispositivo compatible con navegador moderno, quedando el acceso al juego asociado a la sesión del usuario autenticado en la plataforma.

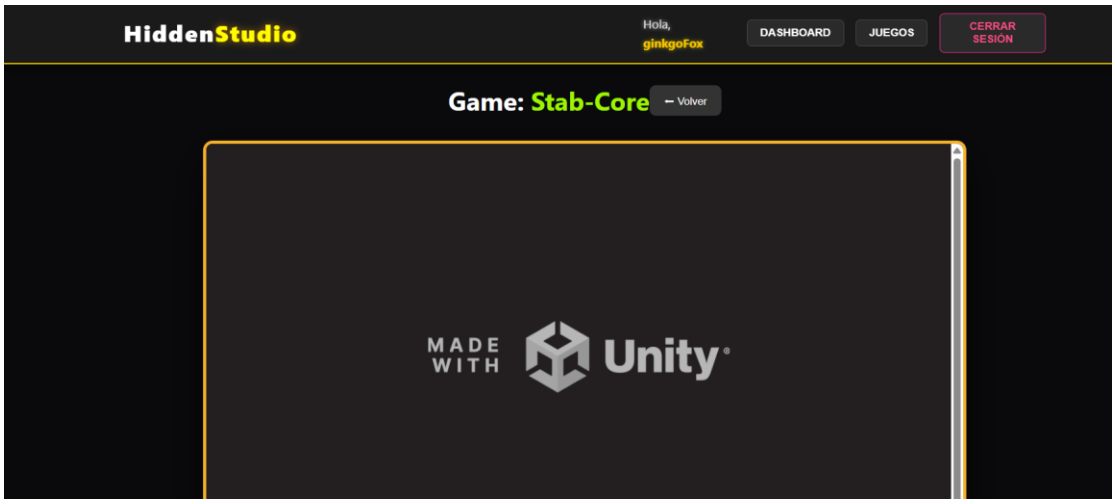
Los juegos exportados en formato WebGL se integran dentro de la aplicación Angular mediante un iframe, el cual incrusta el contenido del videojuego dentro de una vista específica de la plataforma web, manteniendo el layout común de la aplicación.

En el html de detalle de juego tenemos un *div* que actúa como contenedor del juego incrustado, permitiendo controlar su tamaño y posición dentro de la vista:

```
    <button routerLink="/games" class="btn-back">← Volver</button>
  </div>

  <div class="unity-wrapper">
    <iframe
      [src]="gameUrl"
      width="100%"
      height="600"
      frameborder="0"
      allowfullscreen
      loading="lazy">
    </iframe>

  </div>

  <div class="download-section">
```

El nombre de clase asignado a este contenedor lo utilizamos posteriormente en la hoja de estilos CSS para definir aspectos visuales como dimensiones,

márgenes, bordes y comportamiento responsive, garantizando una correcta visualización del juego dentro de la interfaz web:



Como alternativa al juego en navegador, la plataforma ofrece también la posibilidad de descargar el ejecutable del videojuego, permitiendo su ejecución de forma local en el dispositivo del usuario.

Esta opción resulta especialmente adecuada para juegos que requieren un mayor rendimiento, acceso a recursos del sistema o funcionalidades no compatibles con el entorno WebGL del navegador.

Al igual que el enlace al juego WebGL, la API obtiene desde la base de datos la URL del archivo de descarga correspondiente al ejecutable del videojuego.

En este proyecto, ambos recursos se almacenan dentro de la carpeta pública del frontend, organizados en dos subcarpetas diferenciadas:

- games: contiene los archivos exportados en formato WebGL.
- downloads: contiene los archivos comprimidos (.zip) con los ejecutables para PC.

Esta organización facilita la gestión de recursos estáticos y permite diferenciar claramente los formatos de distribución del videojuego.

Al pulsar el botón de descarga, el navegador inicia la descarga del archivo comprimido con el ejecutable del juego para PC:

Ahora viene la parte más importante, los videojuegos de Unity se comunican con el backend mediante peticiones HTTP a la API REST, enviando información como la puntuación obtenida, el tiempo jugado y el progreso del usuario.

Esta comunicación permite actualizar el progreso del usuario en la base de datos, garantizando que la información del juego quede asociada a la cuenta del usuario dentro de la plataforma.

Dentro del componente TypeScript de detalle de juego tenemos configurado listeners que escuchan los eventos enviados desde el videojuego, como actualizaciones de puntuación, incremento del tiempo de juego o avance en el progreso del jugador.

Estos eventos se reciben desde el contexto del iframe y son procesados por el frontend para su posterior envío al backend:

```
puntosActuales = signal<number>(0);
progresoActual = signal<number>(0);

private scoreListener = (event: any) => {
  const puntos = event.detail.puntos;
  this.puntosActuales.set(puntos);
  this.manejarPuntuacionUnity(puntos);
};

private progressListener = (event: any) => {
const progreso = event.detail.progreso;
this.progresoActual.set(progreso);
this.manejarProgresoUnity(progreso);
};
```

Las funciones que reciben estos eventos utilizan métodos definidos en el servicio de la entidad correspondiente, estos servicios se encargan de encapsular la lógica de comunicación con el backend Java, enviando los datos de juego actualizados mediante peticiones HTTP seguras a la API REST:

```
registrarAcceso(juegoId: string, puntos: number, progreso: number): Observable<any> {
  const params = new HttpParams()
    .set('juegoId', juegoId)
    .set('puntos', puntos.toString())
    .set('progreso', progreso.toString());

  return this.http.post(`${this.API_URL}/registrar`, null, {
    params,
    withCredentials: true
  });
}
updateProgress(progress: GameProgressDTO): Observable<GameProgressDTO> {
  return this.http.post<GameProgressDTO>(this.API_URL, progress, {
    withCredentials: true
  });
}
```

El backend es el encargado de validar y procesar la información recibida, decidiendo qué acciones realizar con los datos, como actualizar la puntuación del usuario, registrar el inicio de nueva partida o actualizar el progreso del jugador en la base de datos.

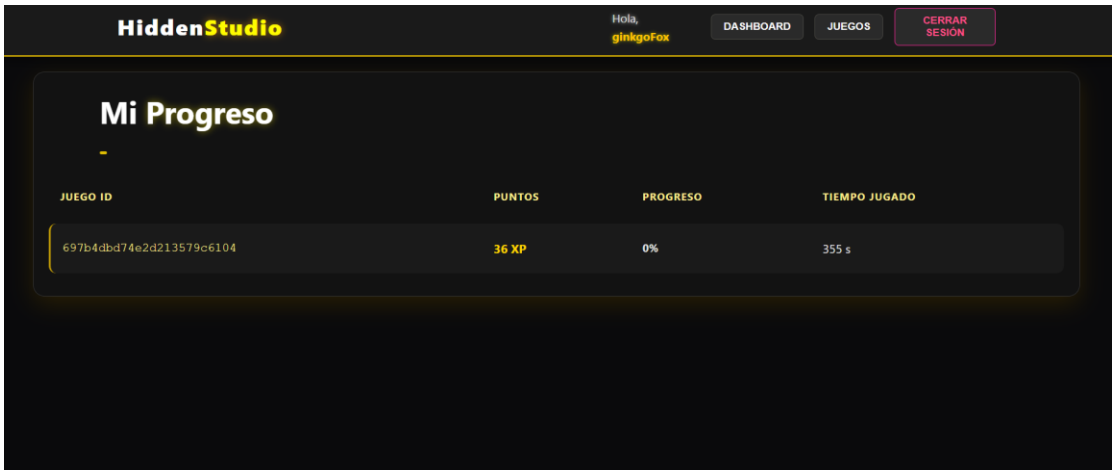
De este modo, se garantiza la coherencia de los datos y se evita que el cliente pueda modificar información sensible de forma directa:

```
public class GameProgressService {
    public GameProgress registrarAcceso(String usuarioId, String juegoId, double progresoSesion, int puntosSesion) {  1 usage
        String key = usuarioId + "-" + juegoId;
        LocalDateTime inicio = sessionCache.get(key);

        double tiempoCalculado = 0.0;
        if (inicio != null) {
            tiempoCalculado = Duration.between(inicio, LocalDateTime.now()).getSeconds();
            sessionCache.put(key, LocalDateTime.now());
        }
        GameProgress gameProgress = gameProgressRepository.findByUsuarioIdAndJuegoId(usuarioId, juegoId)
                .orElseGet(() -> {
                    GameProgress nuevo = new GameProgress();
                    nuevo.setUsuarioId(usuarioId);
                    nuevo.setJuegoId(juegoId);
                    nuevo.setAccesos(0);
                    nuevo.setTiempoJugado(0.0);
                    nuevo.setPuntosMax(0);
                    nuevo.setProgresoMax(0.0);
                    return nuevo;
                });

        if (gameProgress.getProgresoMax() == null) gameProgress.setProgresoMax(0.0);
        if (gameProgress.getPuntosMax() == null) gameProgress.setPuntosMax(0);
        if (gameProgress.getTiempoJugado() == null) gameProgress.setTiempoJugado(0.0);
```

La integración entre la plataforma web y los videojuegos desarrollados con Unity permite ofrecer una experiencia unificada, conectando la cuenta del usuario con su actividad dentro de los juegos y centralizando toda la información en el backend.

Gracias a esta integración, la plataforma puede mostrar estadísticas, registros de partidas y progresos asociados a cada juego desde la vista principal del usuario,

permitiendo un seguimiento de la actividad y fomentando la continuidad de uso de la aplicación:



CONCLUSIÓN

A partir de una idea de negocio definida en el módulo de Sistemas de Gestión Empresarial, se ha diseñado y desarrollado una plataforma web completa basada en arquitectura cliente–servidor, donde el frontend desarrollado en Angular actúa como cliente de una API REST implementada en Java con Spring Boot. Esta arquitectura desacoplada ha permitido una clara separación de responsabilidades, facilitando la escalabilidad, el mantenimiento y la evolución futura del sistema.

El backend estructurado en capas es una buena práctica de diseño y favorece la mantenibilidad del código. Se ha implementado un CRUD completo sobre las principales entidades del sistema, junto con consultas personalizadas y un acceso a datos gestionado mediante Spring Data, garantizando la consistencia, eficiencia y correcta gestión de las transacciones. Además, se ha tenido en cuenta la problemática del rendimiento en el acceso a datos, evitando prácticas ineficientes y aplicando una estrategia de acceso optimizada.

En el ámbito de la seguridad, hemos implementado un sistema de autenticación y autorización basado en Spring Security, JWT y control de roles, permitiendo proteger los recursos del backend y asegurar que cada usuario acceda únicamente a los recursos que le corresponden según su perfil. El uso de DTOs nos permite proteger la información sensible y desacoplar las entidades internas del sistema de los datos expuestos al cliente.

El frontend desarrollado en Angular ha permitido construir una aplicación SPA completa, con enrutado, servicios para la comunicación con el backend, formularios reactivos con validaciones y una interfaz clara y coherente. La correcta integración entre frontend y backend cumple los principios de comunicación entre capas y el uso de servicios REST.

La integración de los videojuegos desarrollados en Unity dentro de la plataforma web ha permitido tanto la ejecución directa en navegador mediante WebGL como la descarga del ejecutable en PC. La comunicación entre el videojuego y el backend mediante peticiones HTTP ha permitido registrar la actividad del usuario, su progreso y sus puntuaciones, ofreciendo una experiencia unificada donde la cuenta del usuario centraliza toda su interacción con la plataforma.

La aplicación desarrollada presenta una solución coherente, escalable y alineada con un caso de uso realista de negocio digital. La arquitectura propuesta sienta una base sólida para futuras ampliaciones, como la incorporación de nuevos

videojuegos, funcionalidades sociales, monetización de contenidos o despliegue en entornos productivos.

Readme Backend

Requisitos Previos

Asegúrate de tener instalado:

- Java JDK 17 o superior.
- Maven (gestor de dependencias).
- Una cuenta activa en MongoDB Atlas (revisa el punto de acceso a base de datos en la memoria).
- Un IDE como IntelliJ IDEA, Eclipse o VS Code.
- Asegura de que están los puertos 8080 y 4200 libres.

Puede tanto hacer pruebas desde un navegador cualquiera o desde postman:

Desde Postman:

- Métodos POST:

  Loguea para obtener el token para consultar las otras direcciones, pega http://localhost:8080/auth/login en la zona de url, en auth elige ningún método y en body pega:

  ```
  {
   "username": "ginkgoFox",
   "password": "shc001"
  }
  ```

  Para crear usuario haz lo mismo, cambia el url a http://localhost:8080/auth/register y pega esto en el body:

  ```
  {
    "username": "nombre",
    "email": "jugador@ejemplo.com",
    "password": "miPasswordSeguro123"
  }
  ```

- Método GET:

  En el resultado que salió del logueo, copia lo que hay dentro de las comillas del token, en forma de autenticación elige bearer token y pega el

token copiado, no necesita nada más, ejecuta la prueba con los siguientes urls:

localhost:8080/user
localhost:8080/game
localhost:8080/me/progress
localhost:8080/inventory

Para las funciones creadas:

- User:

Método	Endpoint	Descripción	Acceso
GET	/user/all_users	Lista todos los usuarios (formato DTO)	**ADMIN**
GET	/user	Obtiene el perfil del usuario autenticado	USER
PUT	/user/add-friend/{friendId}	Añade un amigo por su ID	USER
DELETE	/user/remove-friend/{friendId}	Elimina un amigo por su ID	USER

- Inventory:

Método	Endpoint	Descripción	Acceso
GET	/inventory/all	Lista todos los inventarios globales	**ADMIN**
GET	/inventory	Obtiene los ítems del usuario autenticado	USER

Método	Endpoint	Descripción	Acceso
GET	/inventory/{id}	Obtiene un ítem específico por su ID	USER
POST	/inventory	Registra un nuevo ítem (enviado desde el juego)	USER
PUT	/inventory/{id}	Actualiza datos de un ítem existente	USER
DELETE	/inventory/{id}	Elimina un ítem del inventario	USER

- Progreso de juego:

Método	Endpoint	Descripción	Acceso
POST	/start/{juegoId}	Inicializa la sesión y el contador de tiempo	USER
POST	/registrar	**Desde Unity:**	USER

Método	Endpoint	Descripción	Acceso
		Guarda puntos y progreso actual	
GET	/	Lista el historial de progreso del usuario	USER
GET	/stats	Obtiene tiempo total y accesos totales	USER
DELETE	/{id}	Elimina un registro de progreso por ID	**ADMIN**

- Juegos disponibles:

Método	Endpoint	Descripción	Acceso
GET	/game	Lista todos los juegos disponibles	Público

Método	**Endpoint**	**Descripción**	**Acceso**
GET	/game/{gameId}	Obtiene detalles de un juego específico	Público
POST	/game	Registra un nuevo juego en la plataforma	**ADMIN**
PUT	/game	Actualiza la información de un juego	**ADMIN**
DELETE	/game/delete/{gameId}	Elimina un juego del sistema	**ADMIN**

Readme Frontend

Requisitos Previos

- Node.js: v18.13.0 o superior.
- Angular CLI: Instalado globalmente (npm install -g @angular/cli).
- Backend: El servidor de Spring Boot debe estar en ejecución para que el portal muestre datos.

Desde la herramienta de código (visual studio) inicia el servidor ejecutando: ng serve y navega a http://localhost:4200

Enlace al repositorio

Puede acceder al repositorio de GitHub con todo el proyecto a través del siguiente enlace:

https://github.com/angelaShc/proyecto2xtart